

LLM × ESLint

ハーネスエンジニアリングで AI 出力精度を底上げする

自作 lint ルール / codemod による静的検証強化と
LLM 駆動開発のスケラビリティ

2026 — Tier IV 社内向け

このトークの主張（先に結論）

1. **LLM への自然言語指示は揮発する**。CLAUDE.md やプロンプトに規約を書き続けるのは持続しない。
2. 規律は **lint ルール** に焼き込んで、LLM を**エラー駆動**で動かす。
3. **ESLint は JS/TS でカスタムルールが書ける** → LLM 時代に強い。Biome は plugin 拡張不在で弱い。**oxlint は 2026/03 に ESLint 互換の JS plugins alpha 公開**で有力候補に。
4. **タスクごとに LLM の効きやすさは違う**：lint plugin は手動 0 / 複雑な codemod 実装は人間が本体実装 + テストコード生成 LLM のハイブリッドが現実解。

1. 自然言語 prompt は揮発する

- 「こう書いて / これは禁止」は **その会話だけ** で効く。次のセッション、別の人セッションでは消える。
- CLAUDE.md / agents.md に規約を積んでも：
 - 量が増えるほど **追従精度が落ちる**
 - **守ったかを機械検証する手段がない** ← これが本質
- 結果：レビューワーが毎回同じ指摘 → 時間とトークンを溶かす。

ふんわりした規律は LLM 駆動のスケールに耐えない。
規律は **機械可読な形 = lint ルール** に落とすべき。

2. ハーネスエンジニアリング

ハーネス = LLM の自由出力を、望ましい方向に矯正する治具・検査ライン。

従来

[Issue]

- LLM が実装
- 人間レビュー
(規約遵守 + 設計 + 意図)

レビュー一律速 / 規約指摘がボトルネック。

ハーネス駆動

[Issue]

- LLM が実装
- `eslint --fix`
- 残エラーを LLM に渡す
- `tsc` / `vitest`
- 人間レビュー
(設計 + 意図のみ)

機械のできる検査は機械にやらせる。

ハーネスの構成要素：型システム / **lint ルール** / テスト / フォーマッタ / `cspell`

3. ESLint の基礎 (30 秒復習)

- ESLint = JS/TS のソースを **AST** (抽象構文木) に変換し、**ノードごとに検査関数を呼ぶ** 仕組み。
- カスタムルールは TypeScript で **1 ファイル 数十〜200 行** で書ける。

```
export const myRule: Rule.RuleModule = {
  meta: {
    type: "problem",
    fixable: "code",
    messages: { violate: "..."},
    schema: [
      /* JSON Schema for options */
    ],
  },
  create(context) {
    return {
      ImportDeclaration(node) {
        // node を検査して context.report({ node, messageId, fix }) を呼ぶ
      },
    };
  },
};
```

`fixable` + `fix()` を返すと `eslint --fix` で自動修正 → LLM 駆動ループとの相性◎。

4. なぜ ESLint なのか — 3 ツール比較

ツール	カスタムルール拡張	速度	LLM 時代の評価
ESLint	JS/TS で plugin。AST + scope + 型情報フルアクセス	標準	◎ 既存資産最大。LLM 即生成
Biome	プラグイン機構なし（GritQL ベース限定）	速い	△ 速度は魅力。 規約を機械化できないのが致命的
oxlint	2026/03 に JS plugins alpha 。ESLint v9+ API と 99.5%+ 互換 。tsgolint で型情報も	非常に速い	◎ ESLint 置き換え候補。資産を保てる

主張：plugin 拡張を持たない linter は LLM 駆動時代に戦略的に弱い。
速度 < **規約を機械化できる柔軟性** のレバレッジ。

4.1 oxlint 2026 補足

- **2026/03** に JS plugins alpha 公開。「ESLint プラグインがほぼそのまま動く」段階に。
- **raw transfer** 技術で JS plugin が **86% 高速化** → Rust に近い速度で動く。
- typescript-eslint の **型 aware ルール 61 中 59** を `tsgolint` (typescript-go ベース) でカバー。
- 多くのプロジェクトで **drop-in replacement 候補** として現実的に。

運用上の含意

- 新規に lint 基盤を立てるなら **ESLint 互換の plugin** で書いておけば oxlint へ載せ替え可能。
- 「速度のために規約を諦める (Biome)」ではなく「**互換性を維持しつつ将来高速化 (oxlint)**」が正解。

出典 : [Oxlint JS Plugins Alpha / Type-Aware Linting](#)

事例 A

リポジトリ専用ローカル plugin

```
AutowareEvaluationDashboard/configs/eslint/plugins/
```

5.1 use-alias-import (fixable)

やること：特定ディレクトリ配下を相対 import で参照していたら、**barrel 経由の alias に矯正**。

Before (LLM がよく書く)

```
import { Foo } from "../../components/shared";  
import { Bar } from "../../shared/utils";
```

After (eslint --fix 後)

```
import { Foo } from "~/components/shared";  
import { Bar } from "~/components/shared";
```

- **なぜ必要**：LLM はパス構造を覚えていられず fragile な相対 import を生成。移動・リネームで壊れる。
- **ポリシー**：tsconfig paths には barrel alias のみ登録。サブパスは禁止。
- **規模感**：約 170 行 TS。LLM に仕様を渡せば 1 セッションで生成可能。

5.2 import-layer-order

やること：レイヤーを線形配列で宣言し、**逆方向（低層が高層に依存）を禁止**。

```
layers: [  
  "src/utils", // ← 最も低層（どこからも依存される）  
  "src/core",  
  "src/constants",  
  // ...  
  "src/components/common",  
  "src/components/shared",  
  "src/components", // ← 最も高層  
];
```

- 各ファイルは **prefix が最も長く一致するレイヤー** に所属。その index で大小比較。
- **なぜ必要**：LLM は雑に書くとレイヤー反転（utils → components）を混入させる。
- **効用**：アーキテクチャ違反を **AI に書かせないハード制約**。
- **規模感**：約 200 行 TS、`exemptFiles` で除外可能。

事例 B

コーディングスタイル系の汎用ルール

```
eslint-config-typed/src/plugins/react-coding-style/
```

6. react-coding-style ルール群 (9 本)

「型では表現しきれないが、チームで統一したい記法」を 1 本ずつ AST ルール化。

ルール	何を縛るか
component-name	コンポーネント変数名
component-var-type-annotation	変数宣言の型注釈位置
display-name	React.memo の displayName 必須化
props-type-annotation-style	Props 型を Readonly<{...}> で書く
react-memo-props-argument-name	React.memo の引数名
react-memo-type-parameter	React.memo<Props> の型パラメータ強制
import-style	import * as React from 'react' 形式に統一
use-memo-hook-style	useMemo の宣言スタイル統一
ban-use-imperative-handle-hook	useImperativeHandle の禁止

6.1 抜粋 : display-name

Bad

```
const MyComponent = React.memo(() => <div>Hello</div>);
```

DevTools で Anonymous 扱いされてデバッグ困難。

Good

```
const MyComponent = React.memo(() => <div>Hello</div>);  
MyComponent.displayName = "MyComponent";
```

- AI が出す React コードは「業界の平均」になり、**リポジトリ固有の流儀から静かにズれる**。
- ルール化すれば `eslint --fix` で **強制的に流儀へ寄せられる**。
- 「2 回同じ指摘 → 3 回目はルール化」の文化を作る。

事例 C

LLM が手こずった例 : codemod

```
noshiro-pf/ts-codemod-lib/.../convert-to-readonly.mts
```

7. convert-to-readonly codemod とは

- TypeScript の型定義中の **mutable 型を Readonly 系に自動変換** する `ts-morph` ベース transformer。

```
// Before
type User = {
  tags: string[];
  position: [number, number];
  config: { retries: number };
};

// After (codemod 適用後)
type User = Readonly<{
  tags: readonly string[];
  position: readonly [number, number];
  config: Readonly<{ retries: number }>;
}>;
```

- **なぜ必要** : TS の型はデフォルト mutable。引数として渡した値が呼び先で書き換わるバグが容易に発生 → **Readonly がデフォルト** の規約を型側で強制すれば `tsc` が落ちて検出できる。
- 既存リポジトリ全体に手動適用は非現実的 → codemod 一発で適用。

7.1 規模感と「LLM 効きにくさ」の正直な話

区分	行数
本体 <code>convert-to-readonly.mts</code>	1,482 行
テスト <code>convert-to-readonly.test.mts</code>	3,959 行
helpers	約 250 行

比較：\$5 の ESLint plugin は **170～200 行 / 本**、LLM 駆動でほぼ手動 0。

この **codemod** は **LLM 駆動 100%** では収束しなかった。理由（推察）：

- `ts-morph` の API 表面が広く、ノードごとに **意味論を伴う判断** が連鎖（parens を残す？ `DeepReadonly` で包む？ 中で `readonly` に落とす？）
- **union / intersection / tuple / generic** にまたがる不変条件が多く、リグレッションを起こしやすい
- ESLint のような「**ノードに対する独立検査 + 自動修正**」という疎結合単位に分解できない（`ReadonlyContext` で状態を持ち回す）

7.2 教訓 — タスクごとの「LLM の効きやすさ」

LLM 駆動が効くタスク

- 単一ノードに閉じた検査 (ESLint rule)
- 規約が明確で Before/After が定義しやすい
- 状態が小さい・無い
- **テストケース生成** (網羅性出し)
- 小さなヘルパー関数、型定義

LLM 駆動が苦手なタスク

- AST 全体に渡る意味論的整合性が必要
- 既存ケースを壊さずに新ケースを足す
- API 表面の広いライブラリ (ts-morph 等)
- 状態を持ち回す変換

投資する順序：まず **lint plugin** を量で増やす。codemod は「**骨格は人間 / テストケースは LLM**」のハイブリッドが現実解。
codemod 実装本体は手で書いたが、**テスト 3,959 行の網羅は LLM で大量生成**できた。

8. 推奨ワークフロー

1. Issue / 仕様
2. LLM が実装
3. `eslint --fix` ← ローカル plugin + 汎用 plugin
4. 残ったエラーを LLM に渡す ← 自然言語で説明しない、エラー文だけ
5. `tsc` / `vitest`
6. 人間レビュー（規約遵守は見ない、設計と意図だけ）
7. 同じ指摘が 2 回出たら → ルール化（LLM に書かせる）

運用 Tips

- 人間の prompt 量を増やすのではなく、**ルール本数**を増やす方向に投資。
- ルール追加コストは LLM のおかげで **限界費用ほぼゼロ**。「規約 1 個 = ルール 1 個」。
- ルールに同居する `.test.mts` を必須にし、**ルール自体の壊れも検出**。

9. その先：自前で lint / codemod を増やす

- ESLint で覆えない領域でも、AST パーサさえあれば自前 linter / codemod 構築可能。
- 「ゼロから書く」は従来重い投資だったが、LLM 駆動なら **週末プロジェクト規模**。

想定される自前ツール例

- 社内シナリオ DSL (YAML 等) の静的検証ツール
- 旧 API → 新 API の自動移行 codemod
- リポジトリ横断の禁止 import スキャナ
- protobuf スキーマの後方互換チェッカ

静的検証の自前増殖力こそが、LLM 駆動開発のスケラビリティを決める。

ただし書き換え系 (codemod) はまだ人間の手が要る — 配分は「lint で量 / codemod は質」。

10. まとめ

1. **LLM への指示は揮発する。規律は lint に焼き込め。**
2. **ESLint は plugin 拡張があるおかげで LLM 時代に強い。**
Biome は弱い。 **oxlint は 2026/03 で現実解**になった。
3. **ルール実装は LLM にやらせれば限界費用ゼロ。**
「指示を増やす」より「ハーネスを増やす」。
4. **タスクの性質を見極めよ** : lint plugin は LLM 駆動 100%、
codemod は骨格人間 + テスト LLM のハイブリッド。
テスト生成はどのタスクでも LLM が効く。

→ 今日から : レビューで 2 回同じことを言ったら 3 回目はルールにする。

Thank you

Questions?

参考リポジトリ

- `AutowareEvaluationDashboard/configs/eslint/plugins/`
- `noshiro-pf/eslint-config-typed/src/plugins/react-coding-style/`
- `noshiro-pf/ts-codemod-lib/src/functions/ast-transformers/`